

Documentación final de BeatSound

Lucía Morales Cornejo – 2º DAM (mañana)

ÍNDICE:

1. INTRODUCCIÓN.....	1
1.1. Tecnologías y/o lenguajes usados.....	1
2. ESTUDIO DE MERCADO.....	2
2.1. Empresas y productos parecidos.....	2
2.2. Estructura de departamentos.....	2
2.3. Productos más demandados.....	4
2.4. Oportunidad de negocio.....	4
2.5. Obligaciones laborales, fiscales y riesgos laborales.....	4
2.6. Subvenciones y ayudas.....	5
3. DISEÑO Y PLANIFICACIÓN DEL PROYECTO.....	5
3.1. Análisis de requisitos / planificación.....	5
3.2. Necesidades hardware y software.....	6
3.3. Diseño de casos de uso.....	6
3.4. Base de datos.....	7
4. IMPLEMENTACIÓN Y EVALUACIÓN DEL PROYECTO.....	8
4.1. Fase de desarrollo.....	8
4.1.1. Funcionamiento.....	8
4.1.2. Principales problemas.....	9
4.1.3. Despliegue.....	9
4.1.4. Mejoras de futuro.....	10
4.2. Fase de pruebas.....	10
4.2.1. Pruebas de interfaz.....	10
4.2.2. Pruebas de lógica y filtros.....	11
4.2.3. Pruebas de conectividad.....	11



1. INTRODUCCIÓN

BeatSound es una aplicación móvil tipo red social para Android. El objetivo de realizar esta red social es tener un lugar para dar visibilidad a artistas, donde no compitan en un algoritmo lleno de contenido no relacionado con la industria musical. Además, se trataría de un lugar donde no dependes de tus números como artista para tener visibilidad, el algoritmo te recomienda según tu estilo.

1.1. Tecnologías y/o lenguajes usados

● FRONTEND

- **Dart:** Es un lenguaje de programación desarrollado por Google. Es moderno, tipado y diseñado específicamente para que las interfaces de usuario se muevan de una forma muy fluida.
- **Flutter:** Es el framework que utiliza Dart. Es multiplataforma, lo que resulta una gran ventaja. Con el mismo código, se puede generar una app para Android y para iOS sin cambiar ni una sola línea. Funciona mediante Widgets, todo en la pantalla es un bloque dentro de otro.

● BACKEND

- **Java:** Se trata de un lenguaje robusto, muy seguro y con el que más he programado. Por lo que lo veía muy cómodo para programar la lógica del algoritmo.
- **Spring Boot:** Es un ecosistema para Java que nos permite crear una API REST. Se encarga de escuchar las peticiones del móvil y comunicarse tanto con Firebase como con Cloudinary.

● ALMACENAMIENTO Y DATOS

- **Firebase Firestore:** Guarda la información en documentos y colecciones en formato JSON. Aquí es donde guardo el historial de datos como los usuarios existentes en la app, los IDs de los vídeos que hay, a quien le está gustando cada estilo. Es muy rápida y se actualiza a tiempo real.
- **Firebase Authentication:** Aquí guardamos todos los usuarios registrados en la app junto a su correo, contraseña y hora de creación de la cuenta.
- **Cloudinary:** Las bases de datos no están hechas para guardar archivos pesados como son los vídeos MP4 o MOV. Por eso uso Cloudinary, porque se trata de un servidor optimizado para almacenar vídeos e imágenes. La app pide el vídeo a Cloudinary mediante una URL que se encuentra en el registro del vídeo en Firestore.

● DESPLIEGUE

- **DOCKER:** Permite empaquetar mi código Java, las herramientas necesarias y el archivo de Firebase dentro de un contenedor. Esto asegura que si el código funciona en el portátil, funcionará exactamente igual en los servidores de internet.
- **Render:** Es la nube donde vive mi Spring Boot como contenedor Docker. Render detecta cuando subo el código de mi algoritmo a GitHub, lee las

instrucciones del Dockerfile, compila el proyecto y lo mantiene encendido en internet con una URL segura.

- **DISEÑO**

- **Canva:** Para las ideas mentales que iba teniendo de la aplicación usaba esta herramienta para ir dándole una forma en modo de “demo visual”. Además, para el vídeo de tutorial también uso esta herramienta para darle forma, juntando las imágenes, dándole voz, añadiendo subtítulos...

2. ESTUDIO DE MERCADO

2.1. Empresas y productos parecidos

- **TIKTOK:** Es el rey indiscutible del vídeo en formato vertical corto con scroll. Su diseño de la interfaz y la dependencia total de un algoritmo es lo más parecido a mi proyecto pero lo que le diferencia es que TikTok es generalista, muestra vídeos de muchos temas /humor, cocina, bailes...). Mi proyecto es 100% para contenido musical.
- **YOUTUBE SHORTS / INSTAGRAM REELS:** Se trata de la adaptación de las aplicaciones de Google y Meta para competir con TikTok usando vídeos verticales de corta duración. También son plataformas de consumo de contenido vertical, pero nuevamente mi app se diferencia de estas por ser únicamente de contenido musical.
- **SPOTIFY:** Es la plataforma de streaming musical líder en el mundo. El parecido que tiene con mi proyecto es el algoritmo de recomendación musical para personalizar la experiencia de cada usuario. Pero lo que hace que ambas plataformas se diferencien es que Spotify es de contenido en formato audio, mientras que mi proyecto es en formato vídeo.
- **SOUNDCLOUD:** Es la plataforma más histórica para artistas independientes y emergentes. También tiene un feed vertical tipo TikTok para descubrir temas nuevos. El enfoque en el artista que está empezando y la comunidad musical es lo que más hace que se parezca a mi proyecto. Aún así, la diferencia está en que SoundCloud sigue estando enfocada en el formato de audio tradicional. Mi proyecto ofrece una experiencia más moderna, visual y directa para las generaciones actuales que consumen todo en vídeo.

2.2. Estructura de departamentos

Para que una empresa tecnológica y de entretenimiento musical como BeatSound funcione de manera profesional, necesita estructurarse uniendo dos mundos: el de una empresa de software y el de una plataforma de contenidos musicales.

Si mi proyecto se tuviese que transformar en una empresa real, esta sería la estructura de departamentos clave con la que debería contar:



- **DEPARTAMENTO DE TECNOLOGÍA Y DESARROLLO:** Se trataría del corazón de la empresa. Se encarga de que la aplicación funcione, no se caiga y siga evolucionando. En una fase inicial, este departamento suele trabajar bajo metodologías ágiles como Scrum. Estaría dividido en los siguientes grupos:
 - **Equipo Frontend:** Los especialistas en Flutter y Dart. Se encargan de mantener la aplicación móvil, diseñar nuevas pantallas, animaciones y asegurar que el scroll de vídeos sea fluido tanto en Android como en iOS.
 - **Equipo Backend y Cloud:** Especialistas en Java, Spring Boot y arquitecturas en la nube como Docker o Render. Su trabajo es mantener las conexiones seguras con las bases de datos (Firestore) y el almacenamiento (Cloudinary).
 - **Equipo de datos e inteligencia artificial:** Son los encargados de la magia detrás de la app. Monetizan, analizan y mejorar continuamente el algoritmo de recomendación. Diseñan sistemas productivos para que, si un usuario le gusta por ejemplo el trap, la app sea capaz de refinar si se refiere a drill, trap melódico...
 - **Diseño de producto:** Diseñan la interfaz de usuario y estudian la experiencia del usuario mediante mapas de calor y pruebas para que la app sea intuitiva, adictiva y visualmente atractiva.
- **DEPARTAMENTO DE CONTENIDO Y RELACIONES CON LA INDUSTRIA:** Una plataforma musical no es nada sin música ni artistas. Este departamento actúa como puente entre la tecnología y el talento musical. Se divide en:
 - **Cazatalentos:** Su objetivo es buscar artistas emergentes en plataformas como SoundCloud, Instagram o conciertos locales e invitarlos a subir contenido a BeatSound. Les ayudan a entender cómo funciona la plataforma para maximizar su alcance.
 - **Moderación y control de contenido:** Revisan que los vídeos subidos cumplan con las normativas de derechos de autor y que la calidad de audio y vídeo cumpla con los estándares de la app, evitando spam o contenido inapropiado.
- **DEPARTAMENTO DE MARKETING Y CRECIMIENTO:** Su única misión es conseguir que la aplicación pase de tener unos pocos de usuarios a tener millones, tanto en el lado de los oyentes como en el de artistas.
 - **Adquisición de usuarios:** Crean campañas de publicidad digital en plataformas donde está el público joven para incentivar la descarga de la app.
 - **Gestión de comunidad:** Llevan las redes sociales de BeatSound, crean dinámicas, interactúan con el público y posicionan la marca como el referente de la música emergente.
- **DEPARTAMENTO LEGAL Y FINANZAS:** Al trabajar con música, propiedad intelectual y datos de usuarios en la nube, este área es crítica para evitar demandas y asegurar la viabilidad económica.

- **Equipo legal:** Se encargan de redactar los Términos y Condiciones, cumplir a rajatabla con las leyes de protección de datos y gestionar las licencias de derechos de autor con los artistas para que la música pueda reproducirse legalmente en los vídeos.
- **Finanzas y monetización:** Gestionan los costes de la infraestructura en la nube y planifican las vías de ingresos.

2.3. Productos más demandados

Si analizo las empresas del sector en el que se mueve BeatSound (cruce entre plataformas de streaming musical, redes visuales de consumo rápido y herramientas de distribución para artistas), los productos más destacados son:

- **EN PLATAFORMAS DE STREAMING MUSICAL:** Estas empresas dividen sus productos estrella entre lo que demanda el oyente y el creador. Las suscripciones premium son el producto rey. El usuario paga una cuota mensual a cambio de eliminar anuncios, saltar canciones de forma ilimitada, escuchar música sin conexión a internet y acceder a la máxima calidad de audio.
- **EN REDES VISUALES DE FORMATO VERTICAL:** En estas plataformas, el producto más demandado es el espacio publicitario, diseñado para que no parezca publicidad tradicional.

2.4. Oportunidad de negocio

La verdadera pregunta es, ¿crees que tu proyecto puede ser una oportunidad de negocio?.

Sinceramente, mi respuesta es que Sí. Durante el desarrollo de mi proyecto he querido verificar que esto era real, por lo que he creado contenido en redes hablando de mi proyecto para ver cómo este era recibido. Todo ha sido un total éxito, miles de personas dando las gracias por crear esta comunidad, por pensar en ellos y en qué era lo que necesitaban y nadie ayudaba a mejorar. Muchas marcas, fiestas, artistas con mucho reconocimiento... han estado en contacto conmigo, mandando apoyo, ayudando con la publicidad y la visibilidad de la app... Básicamente creando entre todos una comunidad.

Justo por esto estoy tan segura de que sí, mi proyecto puede ser una oportunidad de negocio.

2.5. Obligaciones laborales, fiscales y riesgos laborales

No existen obligaciones laborales porque no hay contratación de trabajadores ni relación empresa y empleado porque se trata de un proyecto individual, no voy a formar una empresa para comercializar el proyecto. Por ello, no es necesario darme de alta en la seguridad social como autónomo, ya que no voy a sacar beneficios económicos actualmente.

2.6. Subvenciones y ayudas

Si en algún momento consigo que la app salga al mercado, las subvenciones o ayudas que podría pedir son las siguientes:

- **PROGRAMA KIT DIGITAL DEL GOBIERNO DE ESPAÑA:** Destinado a la digitalización y creación de herramientas digitales.
- **AYUDAS CDTI (CENTRO PARA EL DESARROLLO TECNOLÓGICO Y LA INNOVACIÓN):** Financia proyectos innovadores relacionados con desarrollo de software, soluciones digitales e inteligencia de datos.
- **PROGRAMA DE GARANTÍA JUVENIL:** Apoya iniciativas menores de 30 años que desarrollan proyectos innovadores.
- **AYUDAS DEL MINISTERIO DE CULTURA:** Apoya proyectos que fomenten la difusión musical y nuevas plataformas de visibilidad para artistas.

3. DISEÑO Y PLANIFICACIÓN DEL PROYECTO

3.1. Análisis de requisitos / planificación

El objetivo de realizar esta red social es tener un lugar para dar visibilidad a pequeños artistas, ya que entre el contenido del resto de redes sociales, su contenido puede ser opacado al no ser virales.

Para conocer la importancia que el usuario suele darle a la visibilidad de artistas emergentes, los problemas que los artistas suelen tener para darse a conocer, la opinión de los usuarios sobre los algoritmos en otras redes... He creado contenido en redes sociales para saber el apoyo que tendría una app así.

El problema principal que resuelve mi aplicación es la barrera de entrada por algoritmos en la industria musical actual.

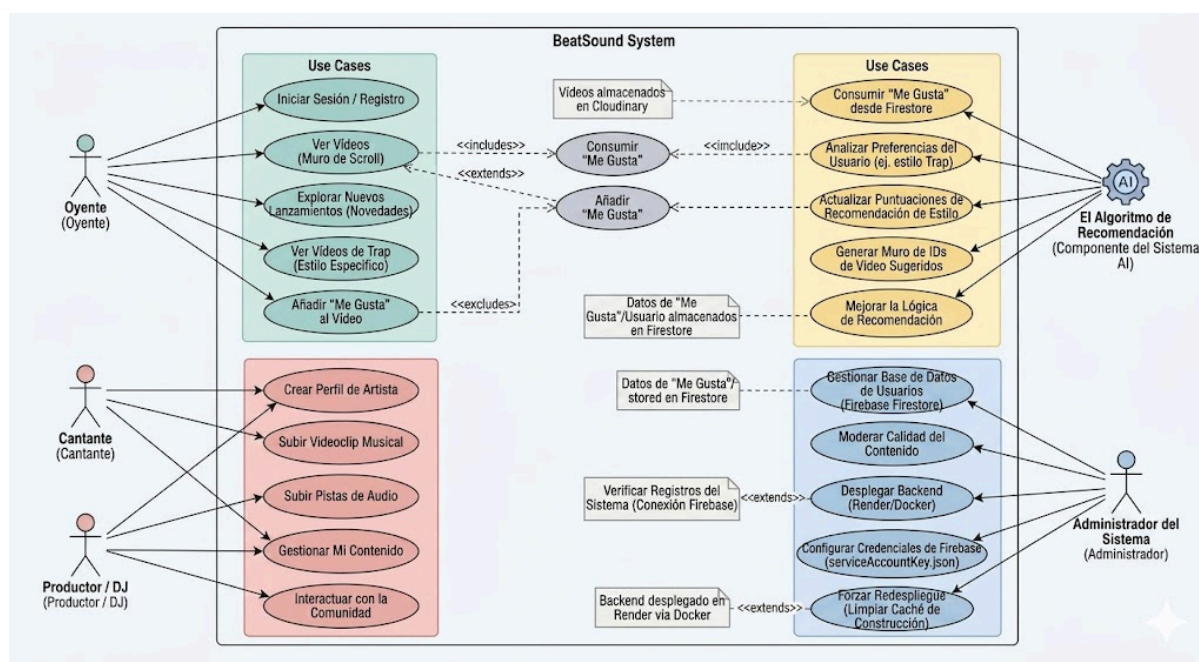
- **Para el artista:** Elimina la necesidad de grandes presupuestos en marketing o de hacerse viral con contenido no musical para poder ser escuchado.
- **Para el oyente:** Resuelve el problema que suelen plantear las plataformas cuando recomiendan solo lo que ya es exitoso en su algoritmo, creando una burbuja de filtro que impide descubrir géneros nuevos.

Para que el sistema sea justo, el algoritmo funciona inicialmente mostrando videos en aleatorio independientemente del estilo musical. Mediante vaya avanzando en likes, dislikes, seguidos o bloqueados, pasa a mostrar primero todos los vídeos del estilo con más puntos tenga en el algoritmo del usuario, luego con el segundo estilo... y así sucesivamente.

3.2. Necesidades hardware y software

- **INFRAESTRUCTURA HARDWARE:** La aplicación está pensada para ser usada en dispositivos móviles, por lo que necesitamos:
 - Móvil Android con acceso a Internet para acceder a la aplicación.
 - Ordenador personal para el desarrollo de la aplicación, la búsqueda de información, la realización de pruebas, llevar el mantenimiento...
 - Servidores en la nube, gestionados a través de Firebase, que se encargan del almacenamiento de datos y la autenticación de usuarios.
- **INFRAESTRUCTURA SOFTWARE:** En cuanto al software necesitaremos las siguientes herramientas:
 - Sistema operativo Android como plataforma principal para ejecutar la aplicación.
 - Como entorno de desarrollo se usará Android Studio para el frontend, Visual Studio Code para el Spring Boot y Firebase Firestore para la base de datos.
 - Cloudinary para subir vídeos en la nube.
 - Render para desplegar el Spring Boot.
 - Conexión a Internet.

3.3. Diseño de casos de uso



3.4. Base de datos

- **COLECCIÓN DE VÍDEOS:**

artista: "Waliyo"
esProductor: true
estilo: "Estudio Tkamelo"
fecha: 24 de abril de 2026 a las 1:00:50 p.m. UTC+2
likes: 0
puntos: 0
titulo: "Video de Beatsound"
url: "https://res.cloudinary.com/dgchm6q2o/video/upload/f_auto,q_auto,w_720/v1777028"

- **COLECCIÓN DE USUARIOS:**

▼ artistas_bloqueados
avatar: "yeidos.jpg"
ciudad: "Murcia"
email: "yeidos@gmail.com"
fecha_creacion: 27 de abril de 2026 a las 3:12:46 p.m. UTC+2
instagram: "https://www.instagram.com/yeidoss?
utm_source=ig_web_button_share_sheet&igsh=ZDNlZDcOMzIxNw=="
▼ lista_seguidos
nombre_artistico: "Yeidos"
rol: "cantante"
spotify: "https://open.spotify.com/intl-es/artist/0UGWnJAVuq3n8l1PkO29Aj?si=IXswTeLBQOak6AvRSwupPw"
tiktok: "https://www.tiktok.com/@yeidoss"
uid: "0QyCZ91UrrSnfGcIDHMcOciU8Pu2"

- **COLECCIÓN DE USUARIO_ESTILO:**

estilo: "trap"
puntos: 20
usuarioUid: "tJ9GhEEUSINPOMQLDHNsgtkoxTq2"

- **COLECCIÓN LIKES:**

artista: "GinovVv"
estilo: "Reggaeton"
uid: "vB2sRK3YTyZ98iGZ1AqFSsRFusQ2"



- **COLECCIÓN POST:**

artista: "Juicy BAE"

ciudad: "Barcelona"

descripcion: "El día 3 al 7 de junio de 2026 nos vemos por el Primavera Sound mi gente de Barcelona ❤️❤️❤️"

fechaPublicacion: 7 de mayo de 2026 a las 12:24:13 p.m. UTC+2

sala: "Primavera Sound"

ticketera: "<https://www.primaverasound.com/es/barcelona>"

título: "Concierto Primavera Sound"

4. IMPLEMENTACIÓN Y EVALUACIÓN DEL PROYECTO

4.1. Fase de desarrollo

Como ya he comentado anteriormente, el objetivo de realizar esta red social es tener un lugar para dar visibilidad a artistas, donde no compitan en un algoritmo lleno de contenido no relacionado con la industria musical. Además, se trataría de un lugar donde no dependes de tus números como artista para tener visibilidad, el algoritmo te recomienda según tu estilo.

4.1.1. Funcionamiento

En la app encontrarás tres roles a elegir cuando te registras:

- **OYENTE:** El rol para cualquier usuario de la app que no suba contenido musical.
- **CANTANTE:** El rol específico para los artistas que quieren darse a conocer por sus canciones. Tienen que subir un vídeo de su galería de su canción junto al estilo musical al que pertenece dicha canción.
- **PRODUCTOR/DJ:** El rol para los artistas que quieren subir sus sesiones o sus bases musicales. Lo que le diferencia del rol de cantante es que, en vez de elegir un estilo concreto, estos ponen el nombre de la sala o estudio desde donde trabajaron el vídeo.

Una vez te registras en la app por primera vez, se te mostrará un pequeño tutorial para entender bien su uso. Ahora encontrarás cuatro apartados en el menú:

- **INICIO:** Es como el para tí de TikTok. Se mostrarán los vídeos en aleatorio según el algoritmo. Si el algoritmo aún no se ha desarrollado (aún no has interactuado con ningún vídeo), los vídeos se muestran aleatoriamente independientemente del estilo musical que tengan. Una vez interactúes con el contenido (like, seguir...), se irán sumando puntos a cada estilo. Cuando vuelvas a entrar a la app, se te mostrará el contenido del estilo que más puntos tenga en tu algoritmo. Es decir, tú le das like a muchos vídeos de Pop, pues tu algoritmo mostrará primero el listado de vídeos de Pop que hay registrados en la app.



- **MAPA:** En este apartado agrupa a cada artista por su provincia, así puedes descubrir artistas de tu ciudad.
- **NOVEDADES:** Aquí se muestran todos los vídeos ordenados desde el último vídeo subido hasta el primero. Puedes usar diversos filtros (sólo artistas, estilos, solo productores/dj).
- **PERFIL:** Dependiendo del rol de tu usuario el perfil mostrará unas opciones u otras. Si tu rol es de oyente simplemente verás los perfiles que sigues y los que tienes bloqueados. Si tu rol es de cantante o productor/dj tendrás un listado de los vídeos que has subido, los perfiles que sigues, los que tienes bloqueados, tus redes y tus post. Puedes acceder a un botón para subir contenido (tanto post para promocionar tus próximos eventos como vídeos).

4.1.2. Principales problemas

Al tratarse de una app bastante completa, he tenido que tener en cuenta muchos detalles que me han ido resultando dificultosos:

- **SERVICIOS EN LA NUBE:** Quería que mi app tuviera los vídeos subidos en algún servicio en la nube (Cloudinary o Firebase). El problema está en que estos servicios tienen una prueba gratuita muy limitada, lo que me limitaba mucho la cantidad de vídeos en la app y hacía que la app se sintiera vacía. Por ello opté por dejar los vídeos que tiene la app dejarlos en local y los que se suban nuevos sí van directamente a Cloudinary. Así, trabajo de una manera híbrida que me permite no limitar mi app a pocos vídeos.
- **DATOS A CAPÓN:** Cuando estuve metiendo los datos de los usuarios y los vídeos que la app tendrá, tuve que meter dato por dato en Firestore. El problema estaba en los nombres porque había muchos casos (solo mayúsculas, mayúsculas y minúsculas alternadas, solo minúsculas, nombres que empiezan por números...). Podía meter un nombre en la tabla de usuarios empezando en minúscula y si luego en sus vídeos lo ponía con la primera en mayúscula, ya no se le vinculaba el vídeo.

El resto de acciones en la app son más visuales, por lo que con la práctica se fue sacando.

4.1.3. Despliegue

Mi proyecto está formado por un backend en Spring Boot y un frontend en Flutter. Para que estos se comuniquen fuera del entorno de desarrollo local, hay que tener una configuración específica de red:

- El servidor debe ser accesible mediante una URL pública desplegando en servicios cloud como AWS o Azure.
- En el código de Flutter, se debe sustituir la dirección del localhost en la configuración de la api (api_service.dart, su conexión con el Spring Boot) por la IP del dominio.

Además, al tener servicios tanto en Cloudinary como en Firestore, hay que configurar esto también.

- **FIRESTORE:** Al configurar el proyecto en la web se crea un .json llamado google-services.json que hay que añadir en el proyecto de flutter. Esto genera una clase firebase_options.dart con toda la configuración. En Spring Boot ponemos la API Key y lo inicializamos en el main.
- **CLOUDINARY:** Al configurar el proyecto en la web, cogemos la Cloud Name, la API Key y el API Secret, todo esto lo ponemos en la configuración de nuestra clase de Spring Boot. Con esto hacemos peticiones HTTPS.

4.1.4. Mejoras de futuro

Para un futuro me gustaría añadir las siguientes funciones:

- **REGISTRO:** Cuando un nuevo usuario se registra, que lo haga a través de su cuenta de Google y se le envíe un correo electrónico con un código de verificación. Así sabemos que los correos que se usan son reales.
- **CONTROL DE CONTENIDO:** Actualmente, en la app se confía que el usuario suba contenido musical pero se puede dar el caso de que algún usuario suba contenido no musical o no apto para cualquier público. Por ello, en un futuro me gustaría implementar algún reporte de contenido o algo relacionado para controlar lo que se sube.
- **LA NUBE:** Actualmente los vídeos están tanto en local como en la nube. Me gustaría que para un futuro todos los vídeos estuvieran subidos en algún servicio en la nube.
- **CONTROL DE ARTISTAS:** Se confía que los artistas son ellos mismos y no se hacen pasar por otros artistas robando su contenido, por ello me gustaría poner alguna verificación (uso de la API de instagram para comprobar que el instagram enlazado es el mismo que la cuenta del usuario que usa la app, ...).

4.2. Fase de pruebas

Para comprobar que todas las funcionalidades de la app funcionan correctamente he realizado diversas pruebas que he organizado por secciones nombrando sólo las más importantes:

4.2.1. Pruebas de interfaz

- **CAMBIO DE VISTAS:** Cuando el usuario pase de una pantalla a otra (ejemplo: Mapa a Novedades), la transición sea fluida y no se muevan los botones, que el tamaño del botón de filtros y su posición sea exacto en ambos cada pantalla para que no parezca que haya un movimiento, que nada se quede trabado...
- **SCROLL HORIZONTAL:** Navegación fluida entre perfiles dentro de la misma provincia en las tarjetas del apartado Mapa. Los avatares se desplazan correctamente y de forma fluida.



- **CARGA DE FOTOS DE PERFIL:** Si el artista no tiene foto se debe mostrar el primer avatar por defecto. Para ello uso una excepción que captura la falta de imagen y muestra el avatar.

4.2.2. Pruebas de lógica y filtros

- **NORMALIZACIÓN DE PROVINCIAS:** Si el usuario pone la provincia entera en minúsculas, o en mayúsculas o con espacios, el mapa debe agruparse igualmente (ejemplo: madrid = Madrid, MADRID = Madrid).
- **FILTRO DINÁMICO POR PROVINCIAS:** Al seleccionar una provincia concreta, sólo se debe mostrar la tarjeta de esta provincia (ejemplo: selecciono en filtros Málaga, sólo sale la tarjeta de los artistas de Málaga). La vista se debe actualizar de inmediato ocultando el resto de tarjetas.
- **EXCLUSIÓN DE VÍDEOS PROPIOS:** El usuario que se registra como cantante o productor/dj, no puede encontrar sus propios vídeos en la navegación del inicio o en las novedades. Para ello el filtro compara en memoria el nombre del artista que está usando la cuenta con el del vídeo que se va a mostrar.

4.2.3. Pruebas de conectividad

- **CONEXIÓN SPRING BOOT:** La APK debe conectar con el backend usando la IP.
- **PERSISTENCIA EN FIRESTORE:** Al registrar un nuevo artista, este debe aparecer automáticamente en el mapa sin reiniciar la app. El stream de Firestore actualiza la interfaz en tiempo real.
- **TIEMPO DE RESPUESTA DEL ALGORITMO:** La recomendación de vídeos debe de tardar menos de dos segundos para no parecer una app trabada.